Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 6:

Inside the xv6 File System

Elaborated: st. gr. FAF-243
Șoimu Ionuț

Verified: asist. univ.
Reițman Patricia

Chișinău – 2026

Contents

1 Lab 6: Track B – Inside the xv6 File System **2**
1.1 Before you start . . . 2
1.2 Part 1. Read what xv6 shows you . . . 2
1.2.1 Inspecting Directory Entries and Inode Properties . . . 2
1.3 Part 2. Write mystat . . . 4
1.3.1 Implementation of user/mystat.c . . . 4
1.3.2 Tracking Hard Links at Runtime . . . 4
1.4 Part 3. Measure the limits . . . 6
1.4.1 Inspecting Filesystem Constants and Disk Geometry . . . 6
1.4.2 Filename Length Boundary Test . . . 6
1.5 Part 4. Write find . . . 7
1.5.1 Implementation of user/find.c . . . 7
1.5.2 Validation on Nested Directories and Hard Links . . . 9

2 Conclusion **10**

1 Lab 6: Track B – Inside the xv6 File System

1.1 Before you start

Purpose and Scope

The file system of the educational operating system xv6 implements the core design patterns of modern Unix storage architectures (such as Linux ext4): on-disk inode structures, directory tables mapping names to inode numbers, reference-counted hard links, indirect block addressing, and crash-recovery logging, all within a concise, readable implementation of a few hundred lines of C.

The primary objective of this laboratory work is to dissect the internal representation of files and directories in xv6. We decode the low-level attributes exposed by xv6 utilities, develop a customized stat utility (`user/mystat.c`) to track inode metadata and hard-link lifecycles, calculate theoretical and architectural filesystem storage limits from definitions in `kernel/fs.h`, and author a recursive directory traversal tool (`user/find.c`) adhering to POSIX filesystem semantics.

Environment Setup and Build Readiness

We verify that the RISC-V cross-compilation toolchain and the QEMU emulator environment are operational, and ensure that the xv6 disk image builds cleanly:

```
➜  ~ cd ~/xv6-riscv
➜  xv6-riscv git:(riscv) which riscv64-linux-gnu-gcc qemu-system-riscv64
/usr/bin/riscv64-linux-gnu-gcc
/usr/bin/qemu-system-riscv64
➜  xv6-riscv git:(riscv) make kernel/kernel fs.img
make: 'kernel/kernel' is up to date.
make: 'fs.img' is up to date.
```

1.2 Part 1. Read what xv6 shows you

1.2.1 Inspecting Directory Entries and Inode Properties

Inside xv6, running `ls` produces a four-column output displaying the entry's filename, file type identifier, inode number, and size in bytes:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35864
echo           2 4 34760
sleep          2 5 34440
args           2 6 34424
forktest       2 7 17704
grep           2 8 43288
init           2 9 35112
kill           2 10 34664
ln             2 11 34456
ls             2 12 41944
mkdir          2 13 34712
rm             2 14 34696
sh             2 15 57072
```

```
stressfs        2 16 35536
usertests       2 17 201520
grind           2 18 50336
wc              2 19 36832
zombie          2 20 34008
logstress       2 21 36680
forphan         2 22 35472
dorphan         2 23 34936
sync            2 24 34112
console         3 25 0
$ mkdir a
$ ls a
.               1 28 32
..              1 1 1024
```

Observe:

1. **What do types 1, 2 and 3 mean (kernel/stat.h)?**
   In `kernel/stat.h`, file types are defined as:
   - `T_DIR` (**1**): Directory – A file containing a list of `struct dirent` entries mapping names to inode numbers.
   - `T_FILE` (**2**): Regular file – An unformatted linear array of data bytes.
   - `T_DEVICE` (**3**): Device – A special character or block device interface (e.g., `console`), identified by major and minor device numbers.

2. **Which inode number does the root directory have, and why are . and .. the same there?**
   The root directory has inode number **1** (`ROOTINO 1` in `kernel/fs.h`).
   In the root directory, both `.` (the current directory) and `..` (the parent directory) point to inode 1 because the root directory is the top-level anchor of the entire filesystem hierarchy; it has no parent directory above it, so by convention its parent pointer `..` points back to itself.

3. **An empty directory a has size 32. A directory entry has a fixed size. How big is one entry, and which two entries does a contain?**
   In xv6, directory entries are defined by `struct dirent` in `kernel/fs.h`:

```
1    struct dirent {
2      ushort inum;
3      char name[DIRSIZ];
4    };
5
```

   Since `ushort` is 2 bytes and `DIRSIZ` is 14 bytes, each directory entry occupies exactly:

$$2 \text{ bytes} + 14 \text{ bytes} = \mathbf{16} \text{ bytes}$$

   A newly created directory contains exactly two mandatory self-referential entries:

   (a) `.` (pointing to directory `a`'s own inode number, 28)
   (b) `..` (pointing to parent root directory inode 1)

   Consequently, its total initial size is:

$$2 \text{ entries} \times 16 \text{ bytes/entry} = \mathbf{32} \text{ bytes}$$

1.3 Part 2. Write mystat

1.3.1 Implementation of user/mystat.c

We implement `user/mystat.c` to invoke the `stat()` system call on each path specified as a command-line argument and print its formatted attributes:

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  if(t == T_DIR)
    return "directory";
  if(t == T_FILE)
    return "file";
  if(t == T_DEVICE)
    return "device";
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;
  if(argc < 2){
    fprintf(2, "usage: mystat <path>...\n");
    exit(1);
  }
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

We register `$U/_mystat\` in `UPROGS` in the xv6 `Makefile`.

1.3.2 Tracking Hard Links at Runtime

We build and run `mystat` inside xv6 to inspect different file types and observe how hard links affect the underlying inode link count:

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 27, links 1, size 0 bytes

$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
```

```
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes

$ cat r2 | wc
48 336 2441
```

Observe:

1. **Paste the outputs after ln and after rm. What changed and what did not?**

   - **Output after `ln README r2`:**

     ```
     README: file, inode 2, links 2, size 2441 bytes
     r2: file, inode 2, links 2, size 2441 bytes
     ```

     Both names share the exact same inode number (**inode 2**) and size (**2441 bytes**). The hard link count (`links`) for inode 2 incremented from 1 to **2**.
   - **Output after `rm README`:**

     ```
     r2: file, inode 2, links 1, size 2441 bytes
     ```

     The name `README` was erased from the root directory table, but the inode number (inode 2) and file content/size (2441 bytes) did not change. The link count decremented back from 2 to **1**. Reading `r2` via `cat r2 | wc` returns the full 2441 bytes unaltered.

2. **Find the line in kernel/sysfile.c (sys_unlink) that decrements the link count. When are the data blocks freed?**

   In `kernel/sysfile.c`, inside `sys_unlink()`, line 244 decrements the link count:

   ```
   ip->nlink--;
   iupdate(ip);
   ```

   When data blocks are freed: Data blocks are not freed during `sys_unlink()` if other directory references or open file handles remain. Instead, freeing occurs inside `iput()` in `kernel/fs.c`. When an inode's in-memory reference count drops to 1 (`ip->ref == 1`, meaning no running processes hold an open file descriptor to it) and its link count is zero (`ip->nlink == 0`), `iput()` calls `itrunc(ip)`:

   ```
   if(ip->valid && ip->nlink == 0){
     ...
     itrunc(ip);
     ip->type = 0;
     iupdate(ip);
   }
   ```

   `itrunc()` frees all 12 direct data blocks and all blocks referenced by the indirect block via `bfree()`, and zeroes out the inode.

3. **xv6 has no symbolic links and no permissions. Look at struct dinode in kernel/fs.h: which fields does an xv6 inode have?**

   In `kernel/fs.h`, `struct dinode` represents the on-disk inode structure:

```
struct dinode {
  short type;              // File type (T_DIR, T_FILE, T_DEVICE)
  short major;             // Major device number (T_DEVICE only)
  short minor;             // Minor device number (T_DEVICE only)
  short nlink;             // Number of links to inode in file system
  uint size;               // Size of file (bytes)
  uint addrs[NDIRECT + 1]; // Data block addresses (12 direct + 1 indirect)
};
```

It contains only 6 fields: `type`, `major`, `minor`, `nlink`, `size`, and `addrs`.
Noticeably absent are:

- No ownership fields (`uid`, `gid`)
- No permission bitmasks (`rwxrwxrwx`, `mode`)
- No access/modification timestamps (`atime`, `mtime`, `ctime`)
- No symbolic link target buffer or symlink type flag

1.4 Part 3. Measure the limits

1.4.1 Inspecting Filesystem Constants and Disk Geometry

We inspect the structural constants in `kernel/fs.h` and examine the disk layout printed by `mkfs`:

```
➜  xv6-riscv git:(riscv) grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|
    define DIRSIZ" kernel/fs.h
5:#define BSIZE  1024 // block size
26:#define NDIRECT  12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14

➜  xv6-riscv git:(riscv) make clean >/dev/null && make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953
    total 2000
```

1.4.2 Filename Length Boundary Test

We test filename truncation in xv6 by creating a directory with a name exceeding 14 characters:

```
$ mkdir averyveryverylongname
$ ls
.              1 1 1024
..             1 1 1024
averyveryveryl 1 29 32
cat            2 3 35864
...
```

Observe:

1. **What does ls show for averyveryverylongname, and which constant explains it?**
   `ls` displays `averyveryveryl`.
   The string `averyveryverylongname` was truncated to exactly 14 characters. This limit is governed by the constant:

`#define DIRSIZ 14` in `kernel/fs.h`

Because each `struct dirent` allocates 14 bytes for `name[DIRSIZ]`, characters beyond the 14th are dropped by `dirlink()`.

2. **Compute the largest file xv6 can hold: 12 direct blocks plus one indirect block that holds block numbers of 4 bytes each, blocks of BSIZE bytes. Show the calculation.**

   - Block size: $BSIZE = \mathbf{1024}$ bytes.
   - Direct blocks: $NDIRECT = \mathbf{12}$ blocks.
   - Size of a block number address: $\text{sizeof(uint)} = \mathbf{4}$ bytes.
   - Capacity of one indirect block:

$$NINDIRECT = \frac{BSIZE}{\text{sizeof(uint)}} = \frac{1024}{4} = \mathbf{256} \text{ data blocks}$$

   - Total maximum block addresses per inode:

$$MAXFILE = NDIRECT + NINDIRECT = 12 + 256 = \mathbf{268} \text{ blocks}$$

   - Maximum theoretical file size:

$$\text{Max File Size} = 268 \text{ blocks} \times 1024 \text{ bytes/block} = \mathbf{274,432} \text{ bytes} \approx \mathbf{268} \text{ KB}$$

3. **From the nmeta line: how many blocks does the whole disk have, and how many are for the log, the inodes, and the data?**

   - **Total disk image size: 2000 blocks** (2,048,000 bytes = 2 MB).
   - **Metadata blocks** (`nmeta 47`)**:**
     - Boot block: **1 block** (sector 0)
     - Super block: **1 block** (sector 1)
     - Log blocks: **31 blocks** (sectors 2–32)
     - Inode blocks: **13 blocks** (sectors 33–45; holds $13 \times 16 = 208$ inodes)
     - Free bitmap: **1 block** (sector 46; tracks up to $1024 \times 8 = 8192$ blocks)
   - **Data blocks:** $2000 - 47 = \mathbf{1953}$ data blocks (sectors 47–1999).

1.5 Part 4. Write find

1.5.1 Implementation of user/find.c

We implement `user/find.c` to recursively walk the directory tree and print any path matching a target basename. It skips unallocated entries (`de.inum == 0`), prevents infinite recursion by ignoring `.` and `..`, closes opened directory descriptors to prevent descriptor exhaustion, and supports an optional `-n` switch for displaying inode numbers:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

void
find(char *path, char *target, int show_inode)
{
```

```
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, O_RDONLY)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }

  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }

  if(st.type != T_DIR){
    close(fd);
    return;
  }

  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';

  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
      continue;

    char name[DIRSIZ + 1];
    memmove(name, de.name, DIRSIZ);
    name[DIRSIZ] = 0;

    // Skip . and .. to prevent infinite recursion
    if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
      continue;

    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;

    // Trim trailing spaces if any
    int len = strlen(p);
    while(len > 0 && p[len - 1] == ' ') {
      p[len - 1] = '\0';
      len--;
    }

    if(stat(buf, &st) < 0){
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }

```

```
    if(strcmp(name, target) == 0){
      if(show_inode)
        printf("%s %d\n", buf, st.ino);
      else
        printf("%s\n", buf);
    }

    if(st.type == T_DIR){
      find(buf, target, show_inode);
    }
  }

  close(fd);
}

int
main(int argc, char *argv[])
{
  int show_inode = 0;

  if(argc < 3){
    fprintf(2, "usage: find <path> <name> [-n]\n");
    exit(1);
  }

  if(argc >= 4 && strcmp(argv[3], "-n") == 0)
    show_inode = 1;

  find(argv[1], argv[2], show_inode);
  exit(0);
}
```

We register `$U/_find\` in `UPROGS` in `Makefile`.

1.5.2 Validation on Nested Directories and Hard Links

We boot xv6 on a clean disk and execute the standard test sequence:

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2

$ find . r2
./a/b/r2
./a/r2
./r2

$ find a b
a/b
```

Stretch Goal: Inode Reporting with -n

We execute `find` with the `-n` flag to display the inode numbers alongside each discovered path:

```
$ find . r2 -n
```

```
./a/b/r2 31
./a/r2 32
./r2 3
```

Observe:

Which of the three r2 files are the same file as cat?

The file `./r2` (with inode number **3**) is the exact same underlying file as `cat` (which also has inode number 3). Although all three files share the same leaf filename (`r2`), `./a/b/r2` (inode 31) and `./a/r2` (inode 32) are distinct, newly created regular files with unique inode entries and independent disk blocks. In contrast, `./r2` was created via `ln cat r2`, which created a second directory entry in `.` pointing to `cat`'s existing inode 3.

2 Conclusion

In this laboratory work, I investigated the foundational mechanics of the Unix file system through the implementation of xv6.

By reading the on-disk structures and decoding the output of `ls`, I explored how a hierarchical directory tree is built out of flat tables of `struct dirent` structures that map human-readable names to numeric inode references. Developing `mystat` made concrete how hard links function: creating a link merely registers an additional name pointing to an existing inode and increments its `nlink` counter, while unlinking removes the directory entry and decrements the counter without freeing data blocks until all active directory links and open file descriptors are released.

Analyzing `kernel/fs.h` illuminated the structural trade-offs of educational file system design: fixed 14-character filename boundaries (`DIRSIZ`), a 1024-byte block size (`BSIZE`), and a 12-direct plus 1-indirect block addressing scheme that limits individual file sizes to approximately 268 KB. Finally, implementing `find` demonstrated the practical mechanics of directory traversal, emphasizing the critical importance of skipping recursive loops (`.` and `..`) and managing process file descriptor limits. Together, these exercises demystified the operating system storage stack from high-level user utilities down to raw disk sectors.